Simulatore di una fabbrica automatizzata

Traccia di progetto C# · Windows Forms · Livello intermedio-avanzato · Progettazione autonoma, senza soluzione

1. Scenario e obiettivo

Una fabbrica dispone di più linee produttive, ciascuna composta da macchinari e robot industriali. Ogni risorsa ha caratteristiche specifiche, consuma energia, può eseguire operazioni e può trovarsi in stati operativi differenti. Il responsabile desidera un software desktop che permetta di configurare l'impianto, avviare le lavorazioni e reagire a guasti o completamenti.

Obiettivo: realizzare un'applicazione C# con una piccola interfaccia Windows Forms. Il progetto deve dimostrare una corretta separazione tra modello a oggetti, logica di simulazione e interfaccia grafica. Non è richiesto il collegamento a macchinari reali.

2. Componenti minimi dell'impianto

- Macchinari: almeno una pressa, un nastro trasportatore e un forno industriale; ciascuno ha parametri e comportamenti specifici.
- Robot industriali: almeno un braccio robotico e un robot di movimentazione; prevedere azioni e limiti differenti.
- Linee produttive: ogni linea contiene più risorse e coordina l'avanzamento di un ciclo di lavorazione.
- Cicli di lavorazione: una sequenza ordinata di fasi con durata simulata e risorsa incaricata; il ciclo può completarsi, arrestarsi o fallire.

3. Requisiti funzionali

- Creare almeno due linee produttive, ciascuna con tre o più risorse di tipologie differenti; visualizzare le risorse associate a ogni linea.
- Avviare, sospendere, riprendere e arrestare una linea, rispettando le condizioni che rendono valide le transizioni di stato.
- Simulare l'esecuzione delle fasi di un ciclo; una fase può partire soltanto quando la risorsa è disponibile e compatibile con l'operazione.
- Tenere traccia, per ogni risorsa, dello stato corrente, del numero di lavorazioni completate e del consumo energetico cumulativo simulato.
- Gestire un guasto su una risorsa: la lavorazione non deve proseguire come se nulla fosse. Consentire una successiva manutenzione/riparazione.
- Mostrare un registro cronologico delle operazioni e delle anomalie, aggiornato durante la simulazione.

4. Regole di dominio

Ogni risorsa deve avere almeno identificativo univoco, nome, potenza nominale e stato. Gli stati ammessi devono essere definiti esplicitamente (per esempio Spenta, Pronta, InEsecuzione, InPausa, Guasta). Le azioni incompatibili con lo stato corrente devono essere rifiutate con un esito gestito e comprensibile all'utente.

Il consumo energetico deve essere calcolato attraverso una regola coerente e documentata, per esempio energia (kWh) = potenza (kW) × durata (ore). Il tempo può essere accelerato o simulato per rendere l'esercizio eseguibile senza attese lunghe.

Vincoli di progettazione software

La valutazione riguarda soprattutto la qualità del modello e la comunicazione tra oggetti, non la quantità di controlli grafici.

5. Requisiti OOP obbligatori

- Classi e incapsulamento: rappresentare le entità del dominio mediante classi con responsabilità chiare; non lasciare che la Form gestisca direttamente le regole di business.
- Classe astratta: introdurre una classe di base astratta per le risorse industriali, contenente caratteristiche comuni e almeno un membro astratto che imponga comportamenti specializzati.
- Ereditarietà e overriding: creare sottoclassi concrete con almeno due implementazioni differenti di comportamento operativo o di calcolo.
- Interfacce: utilizzare almeno due interfacce per capacità trasversali (ad esempio manutenzione, monitoraggio energetico o esecuzione di operazioni), evitando di usarle come copie della classe astratta.
- Polimorfismo: gestire risorse eterogenee attraverso riferimenti al tipo base o alle interfacce, senza sviluppare un grande blocco di controlli sul tipo concreto (if/else o switch).
- Composizione: una linea deve possedere/aggregare le proprie risorse; un ciclo deve organizzare più fasi. Motivare le relazioni uno-a-molti.
- Eventi C#: definire e utilizzare almeno tre eventi del modello, con sottoscrizione e gestione da parte di altri oggetti. La Form deve essere un osservatore, non la sorgente della logica.

6. Eventi richiesti

Evento (nome indicativo)	Quando si verifica	Reazione attesa
StatoRisorsaCambiato	Una risorsa entra in un nuovo stato	Aggiornamento della vista e del registro
GuastoRilevato	Un macchinario o robot rileva un malfunzionamento	Blocco/sospensione della linea interessata e segnalazione
FaseCompletata	Una lavorazione termina correttamente	Avanzamento alla fase successiva, se consentito
CicloCompletato	Tutte le fasi risultano concluse	Aggiornamento conteggi e notifica all’operatore

I nomi sono suggerimenti, non firme obbligatorie. Gli eventi devono trasmettere informazioni utili al destinatario (es. sorgente, stato, messaggio, identificativo della fase). È vietato simulare gli eventi chiamando direttamente metodi della Form dal modello.

7. Comportamenti specifici da differenziare

- La pressa elabora un pezzo con una forza impostata; il forno richiede una temperatura operativa; il nastro movimenta i pezzi con una velocità definita.
- Il braccio robotico esegue operazioni di assemblaggio o prelievo; il robot di movimentazione trasferisce materiale e verifica la capacità massima.
- Almeno una tipologia deve avere un vincolo che impedisca alcune azioni, in modo da mostrare una differenza di comportamento verificabile.

Interfaccia, simulazione e consegna

La WinForms costituisce il pannello operatore: deve comandare il sistema attraverso i suoi oggetti, senza contenerne l’implementazione.

8. Interfaccia Windows Forms minima

- Selezione linea: ComboBox o ListBox per scegliere una linea e visualizzare le sue risorse.
- Stato impianto: DataGridView o ListView con nome, tipologia, stato, numero di operazioni e consumi delle risorse.
- Comandi: pulsanti Avvia, Pausa, Riprendi, Arresta, Simula guasto e Ripara; abilitazione coerente con lo stato selezionato.
- Avanzamento: indicatore dell’attuale fase, percentuale o numero di fasi completate e totale energia consumata.
- Registro eventi: area testuale o lista che mostri timestamp simulato, fonte e descrizione dell’evento.

La simulazione può usare un Timer di Windows Forms o una soluzione equivalente. Se si usa lavoro asincrono, gli aggiornamenti dei controlli devono avvenire in sicurezza sul thread UI. Non è richiesto un database né una grafica avanzata.

9. Scenari di verifica obbligatori

- Avvio regolare: una linea esegue tutte le fasi nell’ordine previsto e genera la notifica finale.
- Guasto durante il ciclo: una risorsa passa allo stato Guasta; la linea reagisce e registra l’anomalia.
- Riparazione e ripresa: dopo la manutenzione, il sistema consente una ripartenza coerente, senza duplicare fasi già completate.
- Operazione non valida: avviare una risorsa guasta o riprendere una linea non sospesa produce un errore gestito.
- Polimorfismo: lo stesso comando operativo invocato su tipologie differenti produce esiti specifici e osservabili.
- Consumi: il riepilogo energetico varia in funzione della potenza delle risorse e del tempo simulato.

10. Elaborati da consegnare

- Progetto Visual Studio compilabile con codice sorgente C# e interfaccia WinForms funzionante.
- Diagramma UML delle classi (anche semplificato) che mostri ereditarietà, interfacce, composizione e associazioni principali.
- Breve documento (1–2 pagine) con le motivazioni delle scelte architetturali, il funzionamento degli eventi e almeno tre scenari di test eseguiti.

11. Vincoli e criteri di valutazione

Ambito	Peso
Modello OO, responsabilità e incapsulamento	30%
Ereditarietà, astrazione, interfacce e polimorfismo	25%
Eventi e coordinamento della simulazione	20%
WinForms e separazione UI / dominio	15%
Gestione anomalie, test e documentazione	10%

Limitazioni: non utilizzare un’unica classe per l’intero progetto; evitare logica di dominio nei gestori Click; non implementare differenze tra macchinari usando soltanto stringhe descrittive o controlli sul nome del tipo. Le classi devono avere responsabilità effettive e verificabili.

Estensioni facoltative: priorità di lavorazione, code di pezzi, manutenzione programmata, storico esportabile in CSV, sensori di temperatura, monitoraggio globale dei consumi.